FP 8 (Haskell 4): Advanced typing & functions

Cael Marquard
mrqcae001@myuct.ac.za
(slides adapted from Dr Sabre Didi)

Computer Science awards for 2026

Please see the announcement by Gary Stewart on the Amathuba site about the CS awards for 2026.

UCT Computer Science Student Awards

Entelect Social Responsiveness Award: For the student making a significant social contribution to the broader IT or South African community using Computing/IT.

Thinkst Leadership Award: For the student exhibiting leadership in student societies, tutoring, mentoring at university or organisational leadership positions related to Computing/IT.

Eligibility: Undergraduate senior students and postgraduate students majoring in Computer Science with an overall rounded average of at least 60% for their Computer Science courses.

Nominations: Nominations can be made by students themselves, other students or members of staff in the Computer Science department.

Submission: Nominators should submit a 1 to 2 page motivation letter and 1 to 3 page CV on the nominee using the following online form: https://forms.gle/MuRKetQkBherUfFy7

Deadline: The deadline for nominations is 2pm, Friday 16 October 2026.

Note: Shortlisted candidates may need to make themselves available for a short interview.

Prize money for the awards is kindly sponsored by Entelect Software and Thinkst.



Aim of this lecture

At the end of this lecture you should:

- Know what “polymorphism” means
- Be able to write & use generic functions and datatypes
- Understand type constructors as functions on types
- Know what a typeclass is, how to derive Show, and what this means

Rationale for polymorphism

So far, we've discussed functions that take **concrete types** and return **concrete types.** That means that an add :: Int -> Int -> Int function can only work on ints, and *nothing* else - not even floats.

```
add' :: Int -> Int -> Int
add' a b = a + b
add' 1 2 -- fine, prints 3
add' 1.0 2.0 -- ERROR! This function takes only Ints
```

But floats can be added just fine… so, **how can we write a function that accepts BOTH?**

Polymorphism and generics

We can do this by making our `add'` function **generic** over the input type. `add'` will then be a **polymorphic / generic** function. (You might remember this from Java—e.g., the ArrayList<T> class is **generic** over some type T)

Polymorphism = poly + morph + ism

- Poly: many
- Morph: form

So, polymorphism = “having many forms”

Declaring a generic function

Let’s rewrite add' to be generic over the input argument. We do this by adding a **type argument/parameter** to the function.

```
add :: Int -> Int -> Int
```

becomes

```
add :: a -> a -> a
```

We can read this as “add is a function with a type parameter **a,** that takes a two **values of type a,** and returns a value of type **a**.

Declaring a generic function (2)

Great! **Now our add function can accept *any* type**. Let's now write the body.

```
add' :: a -> a -> a
add' x y = x + y

error: [GHC-39999]
    * No instance for `Num a' arising from a use of `+'
    * In the expression: x + y
```

Uh oh… **Can anyone tell me what we did wrong?**

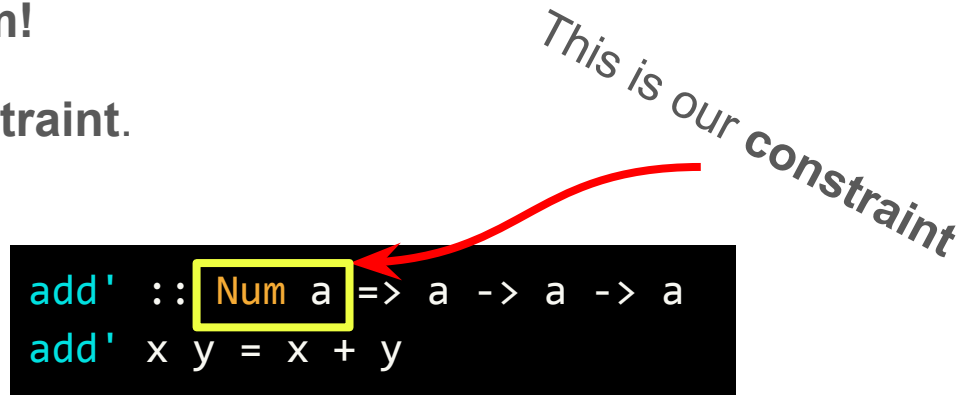
Hint: "our add function can accept ***any*** **type"**

Declaring a generic function (3)

Of course, the problem is that **not just any type can be added with +!** We need to *restrict the domain* of `add'` to numbers only.

Our definition of `add :: a -> a -> a` has a as what's called an **unconstrained type variable.** The only thing this enforces is that both arguments and the return type should all be the same concrete type. It says nothing about **being able to add them!**

We need to add a **constraint**.



Declaring a generic function (4)

With **constraint** in place on add's **type argument**, we can now implement it and use it with both floats and ints successfully!

We now call add' an **overloaded function,** since it has one or more constraints on its type parameters.

```
add' :: Num a => a -> a -> a
add' x y = x + y

add' 1 2 -- returns 3
add' 1.0 2.4 -- returns 3.4
```

[AR: "Ad-hoc Polymorphism" on HaskellWiki]

Type-classes

```
add' :: Num a => a -> a -> a
```

Let's go back to add'. What we just did in our type signature is to **restrict the type parameters of add' to numbers.** But where does this "`Num`" come from?

Num is what is called a **type-class.** This is **not the same as a class in Java or Python. It is more like an `interface`!** (It is also almost exactly equivalent to a `trait` in Rust)

[AR = https://www.haskell.org/tutorial/classes.html]

Type-classes vs classes in OOP

A type-class…

1. Specifies **behaviours**
2. Can implement as many as you wish
3. Inheritance defined at **typeclass** definition OR type definition (can be implemented for foreign types)

A class in OOP…

1. Bundles **data and logic**
2. Often you can only inherit from one
3. Inheritance defined at type definition (cannot inherit “for” a foreign type)

Num as an example typeclass

The type that will implement Num (a type param)
Needed so we can write our fn signatures

```
class Num a where
    (+), (-), (*)       :: a -> a -> a
    negate              :: a -> a
    abs                 :: a -> a
    signum              :: a -> a
    fromInteger         :: Integer -> a
    x - y               = x + negate y
    negate x            = 0 - x
```

Member functions that must be implemented.

Member functions that are provided.

Implementing typeclasses

One of the advantages of typeclasses is being able to **implement it on foreign types.** That is, you can define a typeclasses in one package (e.g. `base`, the haskell standard library), and implement it in another.

`Show` (from `base`) is a very common typeclass to do this with. All it does is specify how to convert your type to a String in order to print it. This is very useful for debugging with the use of the `print :: Show a => a -> IO ()` function (equivalent to `putStrLn show`).

On the next slide, we'll implement `Show` for a custom `StreetAddress` type.

Implementing typeclasses: Show

`Show` only requires us to implement one function: `show :: a -> String`.

There is a tacit contract that our return type should also be a valid Haskell expression (so, we should use this for **debugging purposes** as opposed to pretty-printing).

```haskell
-- Remember, it's legal to name our constructor
-- something other than just the type.
data StreetAddress = Addr {
    number :: Int,
    street :: String,
}

instance Show StreetAddress where
    show addr =
        "Addr { number = " ++ (show $ number addr)
        ++ ", street = " ++ (show $ street addr)
        ++ "}"

x = Addr { number = 1, street = "Main Rd" }
show x -- Addr { number = 1, street = "Main Rd" }
```

Implementing typeclasses: Show (2)

Ok, let's add some more fields to this type…

```
instance Show StreetAddress where
    show addr =
        "Addr { number = "++ (show $ number addr)
        ++ ", street = " ++ (show $ street addr)
        ++ ", postcode = " ++ (show $ postcode addr)
        ++ ", city = " ++ (show $ city addr)
        ++ ", province = " ++ (show $ province addr)
        ...
```

Phew! This is getting really old, really fast!

Deriving typeclasses

Thankfully, there's a solution: instead of implementing some type-classes by hand, we can instead **derive** them.

```
data StreetAddress = Addr
  { number :: Int
  , street :: String
  , postcode :: Int
  , province :: String
  , city :: String
  , country :: String
  } deriving Show

show $ Addr 1 "Main Rd" 7700 "Western Cape" "Cape
Town" "South Africa"
-- "Addr {number = 1, street = \"Main: Rd\",
postcode = 7700, province = \"Western Cape\", city =
\"Cape Town\", country = \"South Africa\"}"
```

Declaring a generic type

It turns out that it's also quite useful to be able to have **data types that are generic** over type parameters too.

To set up an example, let's try represent a user's profile in a social media site.

- A user must **always** have a **name**.
- A user **may** have an **age.**
- A user **may** have their **parent's email** specified (e.g., for <18 year olds)

Here's what that might look like in SQL:

```
CREATE TABLE users (
    name        TEXT NOT NULL,
    age         INTEGER,
    parentEmail TEXT
);
```

Declaring a generic type (2)

Haskell **does not have null values.** It is not possible to assign “null” or “None” to a value of type Int or String, as we have here.

[AR = “Null References: The Billion Dollar Mistake” by Tony Hoare, the inventor of null. Text-version paraphrasing here]

No problem, right? We’ll just use 0 for “no age”, and the empty string “” for “no email”.

```
CREATE TABLE users (
    name        TEXT NOT NULL,
    age         INTEGER,
    parentEmail TEXT
);
```

```
data User = User
    {
        name        :: String
    ,   age         :: Int
    ,   parentEmail :: String
    }

User "Cael" 24 ""
User "Thabo" 16 "mama.ka.thabo@example.com"
User "Haskell Curry" 0 "" -- he is dead
```

Declaring a generic type (3)

OK, let's now write some code to handle users… We are a really annoying and pushy social media company, so let's send spam emails asking if the users have missed us and when they'll be coming back to doomscroll again.

```
sendEmail :: String -> String -> IO ()
-- sends the 1st string to the email address given by the 2nd string
-- the implementation isn't interesting to us here

spamAll :: String -> [User] -> IO ()
-- We will cover what mapM_ does later! It's like map for IO functions, but
-- we throw away the result since it's just ()
spamAll spam users = mapM_ (\user -> sendEmail spam $ email user) users
```

Let's stop and think about *exactly what every part of this bit does.*

Declaring a generic type (4)

If we run our function on our original list of users, what will happen? **What mistake did we make?**

```
sendEmail :: String -> String -> IO ()
-- sends the 1st string to the email address given by the 2nd string

spamAll :: String -> [User] -> IO ()
spamAll spam users = mapM_ (\user -> sendEmail spam $ email user) users

users = [
    User "Cael" 24 "" ,
    User "Thabo" 16 "mama.ka.thabo@example.com",
    User "Haskell Curry" 0 "",
]

spamAll "We miss you!" users
```

Declaring a generic type (5) and the evils of null values

… eventually, we are going to invoke

```
(\user -> sendEmail spam $ email user) (User "Cael" 24 "")
-- ... which equals ...
sendEmail "We miss you!" $ email (User "Cael" 24 "")
-- ... which equals ...
sendEmail "We miss you!" "" -- uh-oh! No email address!
```

… and something bad will happen! **Let's stop and think: how did this happen?**

Declaring a generic type (6)

This is fundamentally a **type error:** we anticipated to have a **non-empty String** in the `sendEmail` function, and we **forgot to handle it being empty**.

We could, of course fix it by filtering. (We'd have to do this in **any** and **every** place we use the user's email. It is even worse if we introduce null emails later in the software development cycle - how do we track them all down?)

```
mapM_ (\user -> sendEmail spam $ email user)
    $ filter (\user -> (email user) /= "") users
```

But… this is a **very preventable error!** In fact, this is exactly what having a type system is *supposed to* help us prevent in the first place!

I have, in fact, already taught you everything you need to fix this error in a better way. Let's stop and think for a bit.

Defining a generic type (7): the Maybe type

The way this is fixed in Haskell is through the **generic Maybe** type:

```
data Maybe a = Nothing | Just a
```

(Note that we write the type parameter **a** just before the = - it is not the same syntax as with functions)

Now we can rewrite our User type in terms of Maybe:

```
data User = User
    {
        name        :: String
    ,   age         :: Maybe Int
    ,   parentEmail :: Maybe String
    }
```

Using the Maybe type

Let’s apply our new definition to our previous spamAll example.

```
data User = User
    {
        name        :: String
    ,   age         :: Maybe Int
    ,   parentEmail :: Maybe String
    }

sendEmail :: String -> String -> IO ()
-- sends the 1st string to the email
address given by the 2nd string
```

```
spamAll :: String -> [User] -> IO ()
spamAll spam users = mapM_
    (\user -> sendEmail spam $ email user)
    users

error: [GHC-83865]
    * Couldn't match expected type: [Char]
                  with actual type: Maybe String
    * In the second argument of `($)', namely
        `email user'
      In the expression:
          sendEmail spam $ email user
```

Using the Maybe type (2)

Great! Our work paid off, and the Glasgow Haskell Compiler (ghc) caught the bug for us **before it happened!** Let's fix it.

Old (error)

```
spamAll :: String -> [User] -> IO ()
spamAll spam users = mapM_
    (\user -> sendEmail spam $ email user)
    users
```

New (fixed!)

```
import Data.Maybe
spamAll :: String -> [User] -> IO ()
spamAll spam users = mapM_
    (\usrEmail -> sendEmail spam usrEmail)
    $ mapMaybe email users
```

```
mapMaybe :: (a -> Maybe b) -> [a] -> [b]                    # Source
```

The `mapMaybe` function is a version of `map` which can throw out elements. In particular, the functional argument returns something of type `Maybe b`. If this is `Nothing`, no element is added on to the result list. If it is `Just b`, then `b` is included in the result list.

Side note: maybe with “” inside

You may have noticed that we could still just construct a `Maybe String` with an empty string inside. This is true! There are ways to close this loophole in Haskell, which we will cover in a later lecture.

[AR = “Make illegal states unrepresentable”]

[AR = “Parse, don’t validate” by Alexis King]

Type constructors are just functions on types

[AR = "Type constructor" on the HaskellWiki]

[AR = "Kinds and Higher-Kinded Types in Haskell" by Gints Dreimanis]

You can think of functions as *mappings between types.* The function `show :: Show a => a -> String` maps an instance of Show to a string.

However, did you know that you can also think of *generic types* as *functions on types?* In the type signature `Maybe Int`, we call `Maybe` the **type constructor**, as it can be seen as a function mapping a type (`Int`) to another type (`Maybe Int`).

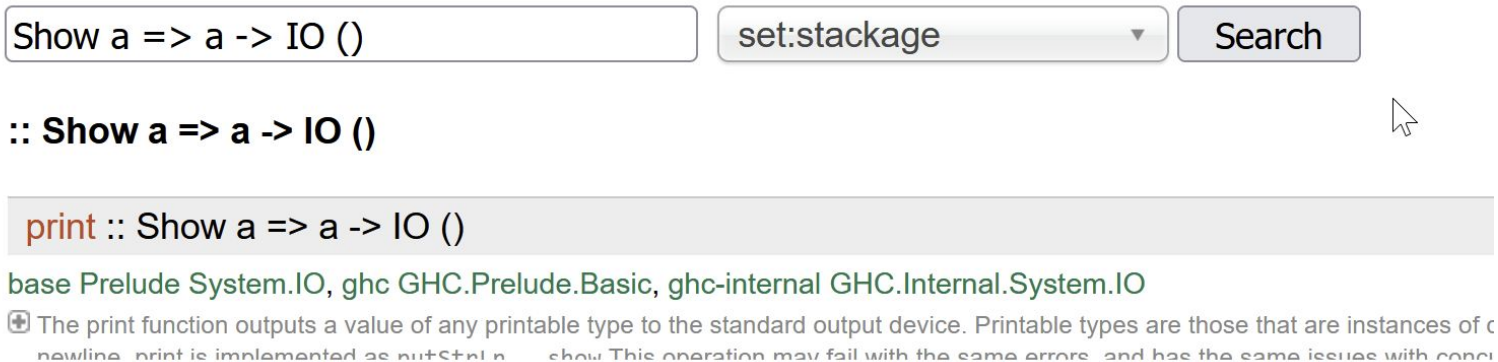
In GHCI, the :kind function is available to show you **the kind of a type** (analagous to :type for values). * means a concrete type, so *** -> * means "mapping type to type".**

```
ghci> :kind Maybe
Maybe :: * -> *
ghci> :kind Maybe Int
Maybe Int :: *
ghci> :kind Int
Int :: *
```

Hoogle

By now, you should understand why it's said that Haskell's type system is very **expressive**.

In fact, it is *so* expressive, that Haskellers often don't search for functions by their names, but by their *type signatures!* This is available using a search engine called Hoogle: https://hoogle.haskell.org/. Cool, right?



Exercises

(please remember that if you use AI to do the following, it no longer has any value. You are only cheating yourself.)

- Reimplement the `sum` function (call it `sum'`) with a generic signature, `sum' :: Num a => [a] -> a`.
- Derive Show for a custom generic type
- Reimplement the `mapMaybe` function: `mapMaybe' :: (a -> Maybe a) -> [a] -> [a]`
- Search for through the functions on Hoogle that map Integers to Strings

Assigned reading & assistance

As revision for today:

- (LYAH) Types and Typeclasses (entire chapter)
- (LYAH) Syntax in Functions (entire chapter)
- (LYAH) Making our own types and typeclasses (entire chapter)

As preparation for next lecture:

- "Foldl and foldr" by Alexis King on the haskell.org blog
- "Why does foldl seem to be harmful despite being tail recursive?" on Stackoverflow (SO > asking AI to explain to you. Fight me)

If you have any queries, email them to me at mrqcae001@myuct.ac.za. I am happy to do office hours by appointment (also by email)